

Cocotb. A co-routine based co-simulation library in Python, it provides an easy way to write complex test benches, which would otherwise need expertise in languages like System Verilog, and allows easy access to signals and writing regression tests.

Extract hierarchy. A tool that does what it says, it is based on Perl-scripting and extracts hierarchy from a set of Verilog source files.

Covered. This Verilog code-coverage analysis tool helps you determine how well your test bench is analysing your code. Your code might have a region that your testing routine never checks, for example, the else-case of an if-else statement, or the code might have some redundant lines that would never be visited.

Code-coverage analysis checks every line of your code, along with checking for state and state transition coverage if the code contains a state machine, toggle coverage and so on, to make sure the code you have written is perfectly functional from all design aspects.

Simbus. It is a set of programs and libraries that support distributed simulation of systems. If your design has major separate components that are connected to each other via a bus, simply interface your device to the bus using the supplied binding libraries. Simbus takes care of inter-device communication and synchronisation for you. It has taken the place of pcisim, a package that supported the development of test benches for peripheral component interconnect (PCI) devices.

GTKWave. It is a waveform viewer to observe value-change-dump (VCD) file that indicates the change in values of a particular variable over simulation runtime. GTKWave supports VCD and lxt file formats.

Breaking down simulation

As perfectly quoted on Iverilog *wikia* page, "Simulation is the process of creating models that mimic the behaviour of the device you are design-

A sample Verilog code for a counter

```
module counter(out, clk, reset);

    parameter WIDTH = 8;
    output [WIDTH-1 : 0] out;
    input clk, reset;
    reg [WIDTH-1 : 0] out;
    wire clk, reset;

    always @(posedge clk)
        out <= out + 1;

    always @reset
        if (reset)
            assign out = 0;
        else
            deassign out;

endmodule // counter
```

A testbench for the counter

```
module test;

    /* Make a reset that pulses once. */
    reg reset = 0;
    initial begin
        # 17 reset = 1;
        # 11 reset = 0;
        # 29 reset = 1;
        # 11 reset = 0;
        # 100 $stop;
    end

    /* Make a regular pulsing clock. */
    reg clk = 0;
    always #5 clk = !clk;

    wire [7:0] value;
    counter c1 (value, clk, reset);

    initial
        $monitor("At time %t, value = %h (%0d)",
            $time, value, value);
endmodule // test
```

ing (simulation models) and creating models to exercise the device (test benches). The simulation model need not reflect any understanding of the underlying technology, and the simulator does not need to know that the design is intended for any specific technology." How clear your code is decides the behaviour of your design. The test bench creates an environment similar to the external world and these are re-used repeatedly at every stage to verify that the design has not undergone changes.

How your code is handled. First, macros in your code, like memory blocks, are pre-processed. The source code and test bench are compiled to check for syntax errors.

The code is then elaborated. If it is a long one, you can break it into different modules, which can be called multiple times in your design, like functions in a C program. There has to be one module, the main module, that is not called anywhere, and this is the root module that calls the others. This hierarchy has to be established and the process of elabo-

ration does exactly this. The process might also include a few optimisations to remove redundant or avoidable codes.

Finally, a code in executable form is generated in the format you specify, and it is this code that is simulated. While simulating, you can provide inputs to your code in different ways: directly in the command line, through the test bench, through a file listing different input cases or you can even have a separate program that generates integer outputs according to the conditions mentioned. These integers can, in turn, be fed as inputs to your code.

Synthesise your code. Following simulation and debugging the code for errors, the next step is to synthesise it. Earlier versions of Icarus Verilog supported synthesis too, but the support has been minimal with recent versions, with more emphasis being given to simulation. You could check out Yosys, an open source tool for synthesis. ●

Priya Ravindran is a technical journalist at EFY